Ders 2 Genişletilmiş Notu: Doğrusal Regresyon, Kayıp Fonksiyonları, Düzenlileştirme ve Lojistik Regresyon

Ders 2 Genişletilmiş Notu: Doğrusal Regresyon, Kayıp Fonksiyonları, Düzenlileştirme ve Lojistik Regresyon

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Sınıflandırmadan Regresyona
- 2. Doğrusal Regresyon Modeli
- 3. Kayıp Fonksiyonları
- 4. Gradyan İnişi ile Ağırlık Güncelleme
- 5. Online Öğrenme Algoritması ve Öğrenme Oranı
- 6. Online, Batch ve Mini-Batch Güncelleme
- 7. Giriş Normalizasyonu
- 8. Genelleme: Underfitting ve Overfitting
- 9. Düzenlileştirme: Ridge, Lasso ve Elastic Net
- 10. Lojistik Regresyon
- 11. Lojistik Regresyonda Ağırlık Güncelleme ve Sayısal Örnek

1. Sınıflandırmadan Regresyona

Hoca derse, bugüne kadar görülen öğrenme yöntemlerinin hepsinin eğitmenli (supervised) yöntemler olduğunu hatırlatarak başladı. Eğitmenli öğrenmede her i. örnek için o örneğin özelliklerinden oluşan bir giriş vektörü ve bu girişe karşılık istenen bir çıkış bilgisi vardır. Bu çıkış bilgisine etiket (label) denir. Örneğin yüz tanıma yapılacaksa giriş bir kişinin yüz özellikleridir, etiket de "bu yüz şu kişiye aittir" bilgisidir. Öğrenme, verilen bu giriş-çıkış çiftlerinden yola çıkarak bilinmeyen bir örnek geldiğinde istenen çıkışın ne olacağını tahmin edebilmektir.

Bugüne kadar görülen yöntemlerin hepsinde bir sınıflandırma vardı. Giriş ayrık (discrete) da olabilir, sürekli (continuous) de; ama çıkış her zaman ayrıktır, yani bir sınıftır (ekrandan, 00:10:00). Bir kişinin fotoğrafı verildiğinde "bu kişi şudur" demek, kişiyi bir sınıf olarak söylemektir. Duygu analizinde bir yazının hakaret içerip içermediğine karar verirken iki sınıf vardır: normal yazılar ve kötü içerikli yazılar. Bir e-postanın spam olup olmadığına karar verirken de iki sınıf vardır: spam ve spam olmayan.

Ama her zaman sınıflandırma istemeyiz. Hocanın verdiği örnekler şunlardır:

- Daha önce görülmemiş bir filmin alacağı ortalama puanın 3 mü, 5 mi olacağını tahmin etmek.
- Bir Instagram kullanıcısının içeriğine bakarak altı ay sonra kaç takipçisi olacağını tahmin etmek.
- Bir semtte yıllardır satılan evlerin özelliklerinden ve fiyatlarından yola çıkarak "3 oda 1 salon, 4. katta" gibi özellikleri verilen bir evin fiyatını tahmin etmek.
- On yıllık yağış, hava sıcaklığı ve toprak durumu verilerinden, bu yılki koşullarla ne kadar mısır hasadı elde edileceğini tahmin etmek.

Bu örneklerde bir sınıf söylenmez, bir değer belirlenir. Öğrenme yine eğitmenlidir: girişler yine ayrık ya da sürekli olabilir, X girişleri için Y çıkışlarında ne istendiği yine öğretilir. Fark, çıkışın sürekli olmasıdır. Ekrandaki ayrım şöyledir (ekrandan, 00:10:00):

- **Classification:** X ayrık ya da sürekli, Y ayrık, yani bir sınıf.
- **Regression:** X ayrık ya da sürekli, Y sürekli. Aranan şey bir $f : X \to Y$ eşlemesidir.

Hoca regresyonun en çok ihtiyaç duyulan ve en çok kullanılan konulardan biri olduğunu söyledi.

1.1 Tek özellikli örnek

Hoca tek özellikli bir örnek çizdi: yatay eksende yıl (2000, 2002, 2004…), düşey eksende satış fiyatı. Eğitim örnekleri çarpı işaretleriyle gösterildi ve bunların arasından geçen bir doğru çizildi (ekrandan, 00:10:00). Amaç, bilinmeyen bir yıl için fiyatın ne olacağını tahmin etmektir. Hoca bunun "son derece kötü bir tahminleme" örneği olduğunu, yalnız fikri göstermek için tek özellik kullandığını belirtti.

Eğitim sonunda elde edilen doğru, örneklerin tam üzerinden geçmez. Bir eğitim örneğinin gerçek değeriyle doğrunun o noktadaki değeri arasındaki dikey uzaklık, o örnekteki hatadır. Yapılmak istenen, verilen x_i girişleri için y_i çıkışlarını üreten öyle bir f fonksiyonu bulmaktır ki bütün n örnek için en iyi eşlemeyi versin, yani hatası en düşük olsun. Doğrusal regresyonda yapılan budur.

2. Doğrusal Regresyon Modeli

Bir örnek, özelliklerinden oluşan bir giriş vektörüyle temsil edilir: $X = (x_1, \dots, x_d)$, burada d özellik sayısıdır (ekrandan, 00:13:36). Film puanı örneğinde özellikler filmin türü (komedi mi), çekildiği yıl, oyuncuları, süresi gibi bilgiler olabilir. Yapılmak istenen, bu özelliklerden çıkışı tespit eden f fonksiyonunu bulmaktır. Hoca bunun da ayırt edici (discriminative) bir öğrenme yöntemi olduğunu söyledi.

Girişler (örneklerin özellik bilgileri) değişmez, istenen çıkışlar da değişmez. O hâlde X'lerden Y'lere gitmeyi öğrenmek, ağırlıklar sayesinde hatayı en küçük yapacak biçimde çıkışı öğrenmek demektir. Bu, perceptron mantığıyla aynıdır: her girişe, o girişin önemini gösteren bir ağırlık bağlıdır.

$$f(x) = w_0 x_0 + w_1 x_1 + \cdots + w_d x_d = \sum_{i=0}^{d} x_i w_i$$

Ekrandaki çizimde $x_1, \dots, x_d$ girişleri $w_1, \dots, w_d$ ağırlıklarıyla bir toplam (Σ) düğümüne girer, çıkış $f(x, w)$ olur. Ayrıca bias girişi vardır: $x_0 = 1$ ve buna bağlı ağırlık w_0 (eşik, θ) (ekrandan, 00:13:36).

2.1 Bias neden $x_0 = 1$ ile gösterilir?

Hoca bunu perceptron'dan hatırlattı. w_0 aslında ayrı bir eşik değeridir. Tasarım ve kodlama sırasında onu ayrı bir şey olarak tutmak yerine normal bir ağırlık gibi ele almak kolaylık sağlar. Öteki girişler örneğin özelliklerine göre değişirken x_0 her zaman 1 alınır. Böylece w_0 da öteki ağırlıklar gibi öğrenme işlemine katılır ve çıkış tek bir toplamla yazılır.

2.2 Hata ve öğrenme

Bir film için izleyicilerin verdiği ortalama puan 4 olsun; bu istenen çıkıştır. Öğrenme sırasında model bu film için 3.5 hesaplamış olsun; bu da modelin gerçek (hesaplanan) çıkışıdır. Hata bu ikisi arasındaki farktır. Öğrenme, bu hatayı en küçük yapacak biçimde ağırlıkları güncelleyerek belirlemektir.

Perceptron'da olduğu gibi ağırlıklar tek seferde tahmin edilemez, yinelemeli (iteratif) olarak belirlenir. Ağırlıklar rastgele değerlerle başlatılır, ardından hatayı azaltacak biçimde güncellenir. Hata belirli bir ϵ değerinin altına düştüğünde, yani yeterince küçüldüğünde öğrenme tamamlanmış sayılır ve model elde edilmiş olur.

3. Kayıp Fonksiyonları

Hata hesaplanırken amaç, hatayı en iyi biçimde en küçük yapacak uygun hata fonksiyonunu kullanmaktır. Hoca "uygun" sözünü şöyle açıkladı: hatanın büyük çıkmasının sebeplerinden biri aykırı değerlerdir (outlier). Model öteki örnekleri iyi öğrense bile aykırı değerler yüzünden hata büyük çıkabilir. Hata hesabı bunları da tolere edebilecek biçimde yapılmalıdır.

Gösterim: n örnek sayısı, y_i istenen (etiketli) çıkış, y_i' modelin hesapladığı (actual) çıkıştır. Ekrandaki dört kayıp fonksiyonu şunlardır (ekrandan, 00:20:00).

3.1 Mean Absolute Error (L1 loss)

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - y_i'|$$

İstenen ve hesaplanan çıkış arasındaki farkların mutlak değerlerinin ortalamasıdır.

3.2 Mean Squared Error (L2 loss)

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - y_i')^2$$

Hoca bunun hata hesaplarında en çok kullanılan fonksiyon olduğunu söyledi. MSE hatayı daha belirgin hâle getirir, yani büyütür: fark büyükse karesi alındığında iyice büyür. Mutlak değerde böyle bir büyüme olmaz. Bunun sonucu şudur: aykırı değer varsa istenen ile hesaplanan arasındaki fark zaten büyük olacaktır, bir de karesi alınınca hata iyice büyür. Ekrandaki not: "OUTLIER varsa hata büyüyor!!".

3.3 Root Mean Squared Error

MSE'nin karekökü alınarak elde edilir. Hoca bunun da çok kullanıldığını, bu fonksiyonlardan herhangi birinin kullanılabileceğini söyledi.

> **Ders sonrası not:** Hoca tahtaya Root MSE'yi $\frac{1}{n}\sqrt{\sum_{i=1}^{n}(y_i - y_i')^2}$ diye yazdı (ekrandan, 00:20:00). Doğrusu $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - y_i')^2}$ biçimindedir; $\frac{1}{n}$ karekökün içindedir, yani RMSE tam olarak $\sqrt{MSE}$'dir.

3.4 Huber loss

Huber loss aykırı değer sorununa duyarlı olacak biçimde tasarlanmıştır. Ekrandaki tanım (ekrandan, 00:20:00):

$$L(y_i, y_i') = \begin{cases} \frac{1}{2}(y_i - y_i')^2 & |y_i - y_i'| \le \delta \\ \delta\left(|y_i - y_i'| - \frac{1}{2}\delta\right) & \text{aksi hâlde} \end{cases}$$

Hatanın mutlak değeri belirli bir eşikten küçükse karesi alınır. Küçük olması, bunun aykırı bir değer değil sıradan bir hata olduğu anlamına gelir. Büyükse hata doğrusal biçimde cezalandırılır. Böylece tek bir aykırı değer yüzünden hatanın büyümesi ve modelin doğru yere yakınsayamaması engellenmiş olur.

Ek bilgi (derste anlatılmadı): Huber loss'un iki parçası $|y_i - y_i'| = \delta$ noktasında aynı değeri ($\frac{1}{2}\delta^2$) ve aynı eğimi verir. Bu yüzden fonksiyon türevlenebilir ve pürüzsüzdür; küçük hatalarda MSE gibi, büyük hatalarda MAE gibi davranır.

4. Gradyan İnişi ile Ağırlık Güncelleme

Hata ölçüsü olarak MSE kullanıldığını düşünelim. n eğitim örneği için hata şöyledir (ekrandan, 00:23:08):

$$J_n(w) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - w^T x_i\right)^2$$

Burada y_i istenen çıkış (desired), $w^T x_i = f(x_i)$ ise modelin hesapladığı çıkıştır (actual). Ağırlıkları hatayı azaltacak biçimde güncellemek için hatanın türevini almak gerekir. Hoca teorik türetmenin ayrıntısına girmeyeceğini, bunu perceptron'dan bildiğimizi söyleyip yalnız sonucu hatırlattı: türev vektörü alınıp sıfıra eşitlenir.

$$\nabla_w J_n(w) = -\frac{2}{n} \sum_{i=1}^{n} \left(y_i - w^T x_i\right) x_i = 0$$

Tek bir örnek için hata $Error_i(w) = \frac{1}{2}(y_i - y_i')^2$ olarak alınırsa, j. özelliğin ağırlığı için güncelleme miktarı şöyle olur (ekrandan, 00:23:08):

$$\Delta w_j = \left(y_i - y_i'\right) x_{i,j}$$

Burada $x_{i,j}$, i. örneğin j. özelliğidir.

Ek bilgi (derste anlatılmadı): Tek örnek hatasında $\frac{1}{2}$ çarpanı kolaylık içindir. $\frac{\partial}{\partial w_j}\frac{1}{2}(y_i - w^T x_i)^2 = -(y_i - w^T x_i)\, x_{i,j}$ olur; 2 ile $\frac{1}{2}$ sadeleşir. Hatayı azaltmak için türevin tersi yönünde gidildiğinden güncelleme $+(y_i - y_i')x_{i,j}$ işaretiyle yazılır.

4.1 Gradyan inişinin sezgisi

Hoca bunun perceptron'da konuşulan gradyan inişi (gradient descent) ile aynı mantık olduğunu söyledi. Yatay eksende ağırlıklar, düşey eksende hata olan bir eğri düşünelim. Ağırlıklar değiştikçe hata azalır. Yolda yerel bir minimum olabilir; amaç hatayı yavaş yavaş azalta azalta global minimuma ulaşmaktır.

4.2 Güncellemenin yönü

Güncelleme kuralında $(y_i - y_i')$ farkı vardır:

- $y_i - y_i' > 0$ ise Δw_j artar. Bu, hesaplanan çıkışın istenenden küçük olduğu, yani çıkışı büyütmek gerektiği anlamına gelir. Çıkışı büyütmek için de ağırlıklar artırılır.
- $y_i - y_i' < 0$ ise Δw_j azalır. Bu durumda hesaplanan çıkış istenenden büyüktür; onu istenen çıkışa yaklaştırmak için ağırlıklar azaltılır.

Hoca bunun çok basit bir mantık olduğunu, perceptron'daki mantığın aynısı olduğunu vurguladı.

5. Online Öğrenme Algoritması ve Öğrenme Oranı

Online gradyan algoritması, ağırlıkların her örnekten sonra değiştirildiği yöntemdir. Öğrenme yinelemeli olduğu için adımlar şöyledir:

- **Ağırlıkları başlatma:** Bias ağırlığı w_0 dahil, her özelliğin etkisini belirleyen ağırlıklar başlatılır. Ağırlıkların nereye gideceği baştan belli olmadığı için genelde -0.5 ile $+0.5$ arasında, sıfıra yakın küçük rastgele sayılarla başlatılırlar. Güncellendikçe kendi yollarını bulurlar.
- **Öğrenme oranını belirleme:** 0 ile 1 arasında bir öğrenme oranı α seçilir.
- **Bir örnek seçme:** Eğitim örneklerinden biri seçilir.
- **Çıkışı hesaplama:** Seçilen örnek için gerçek (hesaplanan) çıkış y_i' bulunur.
- **Ağırlıkları güncelleme:** j. özelliğin ağırlığı, eski değerine öğrenme oranı çarpı hata çarpı o özelliğin değeri eklenerek güncellenir:

$$w_j \leftarrow w_j + \alpha \left(y_i - y_i'\right) x_{i,j}$$

- **Tekrarlama:** Öğrenme tamamlanana kadar, yani hata yeterince küçülene kadar bu adımlar sürdürülür.

5.1 Öğrenme oranının etkisi

Öğrenme oranı büyüdükçe ağırlıklar büyük adımlarla değişir. Ağırlıkların büyük değişmesi hızlı ilerlemek, yani hesaplanan çıkışın istenen çıkışa çabuk yaklaşması demektir. Ama çabuk yaklaşayım derken bir sorun çıkabilir: büyük adımlarla ilerlerken global minimum atlanıp geçilebilir. Bu istenen bir şey değildir, çünkü global minimum atlanırsa hatanın en küçük olduğu yere bir daha dönülemeyebilir.

Bu yüzden öğrenme oranı genelde başlangıçta büyük alınır, çünkü beş iterasyonda istenen sonuca ulaşılamayacağı bellidir. Zamana bağlı olarak değiştirilebilir: örneğin ilk T iterasyonda $\alpha = 0.8$, sonra 0.2. Ya da hiç uğraşmadan baştan küçük bir değer seçilir.

5.2 Öğrenmenin görsel anlatımı

Hoca yıl-fiyat grafiği üzerinde öğrenmenin nasıl ilerlediğini çizdi (ekrandan, 00:32:35). İlk ağırlıklarla elde edilen doğru örneklerden uzaktır. Ağırlıklar güncellendikçe doğru biraz iyileşir, sonra biraz daha iyileşir ve yavaş yavaş örneklere yaklaşır. Son durumda bile bazı örneklerde hesaplanan değer ile gerçek değer arasında bir fark kalır; öğrenme bu farkı en küçük yapacak biçimde doğru denklemini değiştirmektir.

6. Online, Batch ve Mini-Batch Güncelleme

Ağırlıkların ne sıklıkla güncelleneceğine göre üç yol vardır (ekrandan, 00:32:35):

- **Online learning:** Her örnek gösterildikten sonra model, yani ağırlıklar güncellenir.
- **Batch gradient descent:** Bütün örnekler gösterilir, ağırlıklar bir seferde güncellenir. n örnek varsa n örneğin hepsi gösterildikten sonra tek bir güncelleme yapılır.
- **Mini-batch gradient descent:** Belirli sayıda örnekte bir güncelleme yapılır; örneğin "50 örnek gösterdikten sonra ağırlıkları güncelleyeceğim" denebilir.

Hoca genelde mini-batch'in tercih edildiğini söyledi. Çok fazla örnek varsa, örneğin bin örneğin hepsini gösterip ondan sonra güncellemek etkili bir çözüm değildir. Her örnekten sonra güncellemek ise çok yavaştır. Mini-batch'te kaç örnekte bir güncelleme yapılacağı belirlenir ve ona göre ilerlenir. Derin öğrenmede de ağırlık güncellemesi için genellikle mini-batch kullanılır.

7. Giriş Normalizasyonu

Aradan sonra hoca, öteki öğrenme yöntemlerinde olduğu gibi burada da dikkat edilmesi gereken noktalara geçti. Birincisi özellik bilgilerinin, yani girişin normalize edilmesidir. Bu konu k-NN anlatılırken de konuşulmuştu.

Özelliklerden birinin değeri binler mertebesinde, ötekinin 3-5 gibi küçük değerlerde değişebilir. Örneğin bir özellik 2018, 2020 gibi yıl bilgisi, öteki 1, 2, 3 gibi bir puan olabilir. Bunlar normalize edilmeden kullanılırsa etkileri anlamsız biçimde büyük ya da küçük olur. Bunu engellemek için özellikler aynı aralığa getirilir.

Hoca bunun önemini şöyle açıkladı: bir fiyattan söz ediyorsak 1 milyon lira ile 999 bin lira arasında büyük bir fark yoktur. Ama bir puanın 3'ten 4'e ya da 3'ten 6'ya çıkması büyük bir farktır. Değerlerin değişiminin olaya etkisi farklı olduğu için aynı aralığa getirilmeleri gerekir.

7.1 Yapay zekâda hatalı adımlar neden fark edilmez?

Hoca bunu bir karşılaştırmayla anlattı. On sayıyı küçükten büyüğe sıralayan bir algoritma yazıldığında, çıkış sıralı geliyorsa algoritmanın doğru olduğu anlaşılır. Yapay zekâda ise yüzde yüz başarı zaten yoktur: yüz tanımada yüzde 90'lar, duygu analizinde yüzde 85-90 gibi başarılar elde edilir. Bu yüzden bir model tasarlandığında her zaman bir başarı elde edilir ve hatalı yapılan bir adımın neye engel olduğu fark edilemeyebilir. Örneğin normalizasyon yapılmasa da çalışan bir duygu analizi modeli elde edilebilir; ama sistem yeterince iyi öğrenemediği için başarısı düşük olur.

> **Hocanın vurgusu:** Burada iş yalnız doğru kod yazmaktan ibaret değildir; her aşamada yapılması gereken işleri doğru yapmak gerekir. Model oluşturulurken normalizasyon gibi ayrıntıların unutulmadığından emin olunmalıdır.

7.2 Normalizasyon yöntemleri

Ekrandaki iki yöntem (ekrandan, 01:00:33):

- **Min-max normalizasyon:** Bir özelliğin en küçük ve en büyük değeri bulunur, her değer buna göre normalize edilir:

$$x_i' = \frac{x_i - x_{min}}{x_{max} - x_{min}}$$

- **Standardizasyon:** Verinin dağılımının normal (Gauss) dağılım olduğu biliniyorsa kullanılabilir. Ortalama çıkarılıp standart sapmaya bölünür:

$$x_i' = \frac{x_i - \bar{x}}{\sigma}$$

Hoca hangisinin kullanılacağı konusunda min-max normalizasyonun daha çok kullanıldığını söyledi. Farklı normalizasyonlar denenip sonuçları görselleştirilerek ve etkilerine bakılarak da karar verilebilir.

8. Genelleme: Underfitting ve Overfitting

İkinci dikkat edilecek nokta genellemedir (ekrandan, 01:02:45).

Underfitting: Öğrenme tamamlandığında $f(x)$ örneklerin hepsinden epey uzakta kalıyorsa buna underfitting denir. Öğrenme aslında tamamlanmamıştır; iterasyonlara biraz daha devam etmek gerekir. Hoca underfitting'in daha çok tecrübesizlikle yapılan bir şey olduğunu, öğrenmeyi erken kesmenin normalde yapılmaması gereken bir şey olduğunu söyledi.

Overfitting: Model, eğitim verisini çok iyi öğrenecek, örneklerin hepsinin üzerinden geçecek kadar hassas bir çözüm üretir. Tahmin edileceği üzere bu, test başarısını düşürür. Eğitim verisini en iyi biçimde öğrenmek, genellemeye gidememek demektir. Yapılması gereken genel karakteristiği öğrenmektir, ki bilinmeyen bir örnek geldiğinde bu genellemeye göre durumun ne olacağı tahmin edilebilsin. İdeal olan, örneklerin genel eğilimini izleyen ama her örneğe tek tek uymaya çalışmayan bir fonksiyondur.

> **Hocanın vurgusu:** Overfitting çok sık başımıza gelen bir şeydir ve önlenmesi gerekir. Bu tür ayırt edici yaklaşımlarda modelin daha iyi genellemesini sağlamak için kullanılan yollardan biri düzenlileştirmedir (regularization).

9. Düzenlileştirme: Ridge, Lasso ve Elastic Net

Problem şudur: daha iyi genelleyen bir model oluşturmak, overfitting'i engellemek istiyoruz. Ne kadar iyi genellenirse bilinmeyen bir test örneği o kadar doğru tahmin edilir. Düzenlileştirmenin amacı, modelin daha iyi genelleme yapmasına yardımcı olmaktır.

Hesaplanan çıkış $y' = w_0 + w_1x_1 + \cdots + w_nx_n$ ve hata MSE olsun. Düzenlileştirilmiş maliyet fonksiyonu şudur (ekrandan, 01:10:00):

$$\text{Regularized Cost Function} = MSE + \text{Regularized term}$$

9.1 Ridge regresyon (L2 norm)

Hoca bunun en çok kullanılan, varsayılan (default) düzenlileştirme olduğunu söyledi (ekrandan, 01:10:00):

$$\text{Cost} = MSE + \alpha \cdot \frac{1}{2}\sum_{i=0}^{n} w_i^2$$

Buradaki terim L2 norm olarak adlandırılır. Hocaya göre yapılmaya çalışılan şey, örnek sayısı çok arttığında ve model overfitting'e gittiğinde ağırlıkların dengesini korumaktır. Ridge, özelliklerin birbirine göre durumuna bakmadan, ağırlıkların karesini genel olarak maliyet fonksiyonuna ekler.

> **Ders sonrası not:** Hoca α'nın 0 ile 1 arasında değiştiğini söyledi ve tahtaya $0 < \alpha < 1$ yazdı. Büyük olasılıkla bu, derste kullanılan pratik aralıktır; genel olarak düzenlileştirme katsayısı $\alpha \geq 0$ herhangi bir değer olabilir ve 1'den büyük de seçilebilir.

9.2 Lasso regresyon (L1 norm)

Bazı özellikler daha önemliyse ve önemli olmayan özellikler elenmek isteniyorsa Lasso regresyon kullanılır:

$$\text{Cost} = MSE + \alpha \sum_{i=0}^{n} |w_i|$$

Bu terime L1 norm denir. Lasso'da pek çok ağırlık sıfır olur ve önemli olmayan özellikler elenir ("Eliminates unimportant features", ekrandan, 01:10:00). Bir özelliğin ağırlığı sıfır olursa o özelliğin hiçbir etkisi kalmaz. Yani Lasso ile bir özellik seçimi de yapılmış olur. Hoca özellik seçimini ayrıca anlatacağını, bunun ayrı bir konu olduğunu söyledi.

> **Ders sonrası not:** Tahtada Lasso'nun terimi $\sum |w_i|^2$ diye yazılmış görünüyor (ekrandan, 01:08:06), ama hoca aynı terimi "L1 norm" diye adlandırdı. L1 terimi $\sum |w_i|$ biçimindedir; kare alınırsa terim Ridge'deki L2 terimiyle aynı olur ve ağırlıkları sıfıra götürme özelliği kaybolur.

9.3 Elastic Net

Elastic Net, L1 ve L2 düzenlileştirmenin karışımı gibidir (ekrandan, 01:10:00):

$$\text{Cost} = MSE + \alpha\,\frac{1-r}{2}\sum_{i=0}^{n} w_i^2 + \alpha\, r \sum_{i=0}^{n} |w_i|$$

Hoca bunu formül üzerinden gösterdi:

- $r = 1$ alınırsa $(1 - r)$ çarpanı sıfır olur, birinci terim kaybolur ve geriye L1 terimi kalır: bu Lasso regresyondur.

- $r = 0$ alınırsa ikinci terim sıfır olur, $(1 - r) = 1$ olduğu için geriye L2 terimi kalır: bu Ridge regresyondur.

Yani r değerine bağlı olarak Elastic Net, Ridge ile Lasso arasında ayarlanabilir.

> **Ders sonrası not:** Tahtada Elastic Net'in L1 terimi de $\sum |w_i|^2$ diye yazılmış görünüyor (ekrandan, 01:10:00); Lasso için yapılan düzeltme burada da geçerlidir.

Hoca düzenlileştirmeye ileride tekrar döneceğini söyledi. Eğitmensiz öğrenme anlatıldıktan sonra, derin öğrenmeye ayrılacak haftalarda bias, variance ve düzenlileştirme konularından yeniden söz edecek.

10. Lojistik Regresyon

Hoca doğrusal regresyonu şöyle bağladı: sınıflandırma değil de gerçek bir değer hesaplanmak isteniyorsa regresyon kullanılır. Ardından adı regresyon gibi görünen ama aslında regresyonla ilgisi olmayan bir konuya geçti: lojistik regresyon. Adında regresyon geçtiği için sürekli bir çıkış hesaplıyormuş gibi görünür, ama lojistik regresyon aslında bir sınıflandırma yöntemidir.

Normal sınıflandırmada kesin bir sonuç verilir: örneğin doküman sınıflandırmada "bu fizikle ilgili bir dokümandır", "bu turizmle ilgili bir dokümandır" denir. Lojistik regresyonda ise bir değer verilir: "0.9 puanla şöyledir", "0.7 puanla böyledir". Çıkış kesin bir sonuç yerine pürüzsüz (smooth) bir sonuçtur, yani bir skordur. Hoca lojistik regresyonun bu yüzden tercih edilen ve başarısı yüksek yöntemlerden biri olduğunu söyledi.

10.1 Skor mu, kesin sınıf mı?

Hoca bankacılık gibi sektörlerin bu skoru çok sevdiğini anlattı. Örneğin bir dolandırıcılık tespiti (fraud detection) probleminde "bu dolandırıcılıktır" ya da "değildir" demek yerine "bu 0.8 olasılıkla dolandırıcılıktır" denmesini tercih ediyorlar.

Hoca kendi adına farklı düşündüğünü de söyledi: bir şeyin olma durumuna ait skor iyi bir şey olsa da kendi çalışmalarında kesin sınıflandırma sonucunu tercih ediyor. Sebebi şu: sistem "0.8 dolandırıcılıktır" dediğinde gerçekten dolandırıcılık olduğuna ikna olmak için 0.85 mi, 0.9 mu gerektiği yine belirsizdir. Bu kısım yine tecrübe ister, insan faktörü işin içine girer.

10.2 Model

Burada yine ikili sınıflandırma (binary classification) düşünülür; lojistik regresyon da ayırt edici yöntemlerden biridir. İki sınıfa ait örnekler vardır ve amaç bu iki sınıfı birbirinden en iyi ayıran karar sınırını belirlemektir. Tahtadaki çizimde bir sınıf çarpılarla, öteki halkalarla gösterilmiş, aralarındaki karar sınırı $w^T x = 0$ olarak yazılmıştır (ekrandan, 01:17:57).

Bunun yolu artık bilinmektedir: girişler ve çıkışlar değiştirilemeyeceğine göre, verilen girişler için istenen çıkışlara ulaşmayı ağırlık değerleriyle öğrenmeye çalışılır. Karar sınırı ağırlık değerleri sayesinde belirlenir.

Model şöyledir (ekrandan, 01:17:57). n özellik vardır; ayrıca $x_0 = 1$ olan bir giriş daha vardır. Hoca bunun gerçek bir özellik olmadığını, eşik seviyesini belirlemek için kullanıldığını hatırlattı. Girişler ağırlıklarla çarpılıp toplanır:

$$z = \sum_{i=0}^{n} x_i w_i$$

Ardından bu toplam bir fonksiyondan geçirilerek çıkış $f(x, w)$ elde edilir. Lojistik regresyonda kullanılan bu fonksiyona lojistik fonksiyon (logistic function) denir. Hoca bunun çok katmanlı perceptron'da kullanılan sigmoid fonksiyonundan başka bir şey olmadığını söyledi:

$$g(z) = \frac{1}{1 + e^{-z}}$$

10.3 Hard switching ve smooth switching

Tahtada iki geçiş biçimi karşılaştırıldı (ekrandan, 01:17:57):

- **Hard switching:** Basamak fonksiyonu. Belirli bir değerin altında çıkış 0, üstünde 1 olur; geçiş anidir.
- **Logistic (smooth) switching:** Sigmoid. Çıkış 0'dan 1'e pürüzsüz biçimde geçer.

Lojistik fonksiyonun çıkışı 0 ile 1 arasında bir değerdir, yani bir skordur.

10.4 Karar kuralı

Skor, sınıf olarak değerlendirilmek istenirse bir eşikle karşılaştırılır. Verilen giriş değerleri için üretilen sonuç 0.5'ten ya da seçilen bir eşik θ'dan küçükse çıkış 0, değilse 1 kabul edilir. Ekrandaki yazım (ekrandan, 01:20:02):

$$\text{decision}(x) = \begin{cases} 0 & \text{eğer } P(y=0 \mid x) < 0.5 \text{ ya da } < \theta \\ 1 & \text{aksi hâlde} \end{cases}$$

> **Ders sonrası not:** Tahtada koşul $P(y=0 \mid x) < 0.5$ diye yazılmış. Sigmoid çıkışı $f(x) = P(y=1 \mid x)$ olarak yorumlanır; doğru kural "$f(x) = P(y=1 \mid x) < 0.5$ (ya da $< \theta$) ise 0, değilse 1" biçimindedir. Hocanın sözlü anlatımı ("üretilen sonuç 0.5'ten küçükse çıkış 0") bu doğru kurala uyuyor.

11. Lojistik Regresyonda Ağırlık Güncelleme ve Sayısal Örnek

$f = f(x, w)$ lojistik fonksiyon olmak üzere çıkış $[0, 1]$ aralığındadır. Ağırlıklar şöyle güncellenir (ekrandan, 01:25:36):

$$w_j(t+1) = w_j(t) + \alpha_i \left[y_i - \frac{1}{1 + e^{-\sum_{j=1}^{n} x_{i,j}\, w_j}} \right] x_{i,j}$$

Burada w_j, j. özelliğin ağırlığıdır; y_i, i. örneğin istenen çıkışıdır; köşeli parantezin içindeki kesir y_i', yani sigmoid ile hesaplanan çıkıştır; $x_{i,j}$ ise i. örneğin j. özelliğidir.

Hoca kullanılan öğrenme algoritmasının doğrusal regresyondaki online gradyan algoritmasının tamamen aynısı olduğunu söyledi. Tek fark, y''nin hesabında sigmoid fonksiyonunun kullanılmasıdır. Bunun dışında mantık aynıdır; yine online, mini-batch ya da batch öğrenmeden biri kullanılabilir.

> **Ek bilgi (derste anlatılmadı):** Lojistik regresyonda büyük olasılıkla en yaygın tercih, MSE yerine çapraz entropi (log loss) kaybıdır. Bu kaybın türevi alındığında, doğrusal regresyondakiyle aynı biçimde $(y_i - y_i')x_{i,j}$ güncellemesi elde edilir. Derste yazılan güncelleme kuralının doğrusal regresyondakine bu kadar benzemesinin sebebi budur.

11.1 Duygu analizi örneği

Hoca bir duygu analizi (sentiment analysis) örneği verdi. Tweetler için dört özellik tanımlandı (ekrandan, 01:22:00):

- x_1: tweetteki olumlu kelime sayısı.
- x_2: tweetteki olumsuz kelime sayısı.
- x_3: zamir sayısı (I, you, me…).
- x_4: tweetteki toplam kelime sayısı.

Her örnek bu özelliklerle verilir. Örneğin olumlu kelime sayısı 5, olumsuz kelime sayısı 1, zamir sayısı 4, toplam kelime sayısı 6 olan bir örnek $(5, 1, 4, 6)$ vektörüyle gösterilir ve çıkışı 1 (olumlu) olarak etiketlenir. İki sınıf için örnekler bu şekilde verilir. Eğitimden sonra bilinmeyen bir örnek verildiğinde model 0 ile 1 arasında bir değer üretir. Değer 1'e yakınsa tweet olumlu, 0'a yakınsa olumsuz kabul edilir.

11.2 Gösterim üzerine soru-cevap

Soru: Güncelleme kuralında x_i ile çarptık; özelliklerin hepsi için ayrıca toplama mı yapacağız?

Cevap: Hoca önce tahtadaki gösterimin karıştığını fark etti ve düzeltti. Sigmoidin içindeki toplam özellikler üzerindendir ve bunun için j indisi kullanılmalıdır: $\sum_{j=1}^{n} x_{i,j} w_j$. Dışarıdaki i indisi örneği gösterir ("ith sample"). Kuralın sonundaki çarpan da güncellenen ağırlığın bağlı olduğu özelliğin değeridir, $x_{i,j}$ ("jth feature"). Yani w_j güncellenirken, i. örnek için hesaplanan hata j. özelliğin o örnekteki değeriyle çarpılır (ekrandan, 01:25:36).

Soru: α_i neden indisli yazıldı?

Cevap: Öğrenme oranı her adımda değiştirilebileceği içindir. Bütün eğitim boyunca tek bir α (örneğin 0.2) kullanılabileceği gibi, örneğin ilk 500 iterasyonda $\alpha = 0.9$, sonrasında $\alpha = 0.1$ da alınabilir.

11.3 Sayısal örnek

Hoca gösterimi netleştirmek için iki özellikli bir örnek çözdü (ekrandan, 01:28:18). Model: $x_0 = 1$ bias girişi (w_0 ağırlığıyla), x_1 ve x_2 özellikleri (w_1 ve w_2 ağırlıklarıyla). Online öğrenmede ikinci örnek gösterilmiş olsun.

Verilen:

- İkinci örneğin özellikleri: $x_1 = 1$, $x_2 = 4$; bias girişi $x_0 = 1$.
- O anki ağırlıklar: $w_0 = 0.2$, $w_1 = 0.1$, $w_2 = 0.3$.

Çıkışın hesabı:

$$z = w_0 x_0 + w_1 x_1 + w_2 x_2 = 0.2 \cdot 1 + 0.1 \cdot 1 + 0.3 \cdot 4 = 0.2 + 0.1 + 1.2 = 1.5$$

$$y' = \frac{1}{1 + e^{-(0.2+0.1+0.3 \cdot 4)}} = \frac{1}{1 + e^{-1.5}}$$

Hoca çıkışı bu ifadeyle bıraktı. Sayısal değeri $y' \approx 0.818$'dir (ara adım eklendi).

Güncelleme: Online öğrenmede bu örnek gösterildikten hemen sonra ağırlıklar güncellenir. Örneğin w_1 değiştirilmek istenirse:

$$w_1 \leftarrow w_1 + \alpha \, (y_2 - y_2') \, x_1$$

Yani w_1'in yeni değeri, eski değeri artı öğrenme oranı çarpı istenen çıkış ile hesaplanan çıkış arasındaki fark çarpı x_1 özelliğinin değeridir. İstenen çıkış y_2 ve öğrenme oranı derste sayı olarak verilmediği için güncellemenin sonucu hesaplanmadı.

Örnek Sorular

Soru 1 (ekrandan, 01:28:18): Bir lojistik regresyon modelinde $x_0 = 1$, $x_1 = 1$, $x_2 = 4$ ve $w_0 = 0.2$, $w_1 = 0.1$, $w_2 = 0.3$ olsun. Bu örnek için modelin çıkışı y' nedir? w_1 nasıl güncellenir?

Derste verilen çözüm: $y' = \frac{1}{1+e^{-(0.2+0.1+0.3 \cdot 4)}}$ ve $w_1 \leftarrow w_1 + \alpha(y_i - y_i')x_1$.

> **Çözüm (derste verilmedi):** $z = 1.5$, $y' = \frac{1}{1+e^{-1.5}} \approx 0.8176$ (python ile hesaplandı).
> Eşik 0.5 alınırsa örnek 1 sınıfına atanır. İstenen çıkış ve öğrenme oranı derste verilmediği için güncellemenin sayısal sonucu bulunamaz.

Soru 2 (derste sözlü, 01:09:23): Elastic Net maliyet fonksiyonunda $r = 1$ ve $r = 0$ alınırsa hangi yöntemler elde edilir?

Derste verilen çözüm: $r = 1$ ise $(1 - r) = 0$ olur, L2 terimi kaybolur ve Lasso regresyon elde edilir. $r = 0$ ise L1 terimi kaybolur, L2 terimi kalır ve Ridge regresyon elde edilir.

Soru 3 (derste sözlü, 01:23:45): Lojistik regresyonun ağırlık güncelleme kuralında i ve j indisleri neyi gösterir?

Derste verilen çözüm: i örneği, j özelliği gösterir. Sigmoidin içindeki toplam özellikler üzerindendir: $\sum_j x_{i,j} w_j$. Güncellenen w_j ağırlığı, i. örnekteki hata ile o örneğin j. özellik değeri $x_{i,j}$'nin çarpımı kadar değişir.

Dersin Toparlaması

Bu derste eğitmenli öğrenmenin ikinci büyük türü olan regresyon ele alındı. Sınıflandırmada çıkış bir sınıfken regresyonda çıkış sürekli bir değerdir. Doğrusal regresyonda çıkış, özelliklerin ağırlıklı toplamıdır ve öğrenme, perceptron'da olduğu gibi, hatayı en küçük yapan ağırlıkları gradyan inişiyle yinelemeli olarak bulmaktır. Hata MAE, MSE, RMSE ya da Huber loss ile ölçülebilir; MSE en yaygın olanıdır ama aykırı değerlere duyarlıdır. Ağırlıklar her örnekten sonra (online), bütün örneklerden sonra (batch) ya da belli sayıda örnekte bir (mini-batch) güncellenebilir; pratikte mini-batch tercih edilir. Girişlerin normalize edilmesi ve overfitting'in önlenmesi iyi bir model için şarttır; Ridge, Lasso ve Elastic Net düzenlileştirmeleri genellemeyi iyileştirmenin yollarıdır ve Lasso aynı zamanda özellik seçimi yapar.

Dersin ikinci kısmında lojistik regresyon anlatıldı: adına rağmen bir sınıflandırma yöntemidir; ağırlıklı toplamı sigmoid fonksiyonundan geçirerek 0 ile 1 arasında bir skor üretir ve bu skor bir eşikle sınıfa çevrilebilir. Ağırlık güncelleme mantığı doğrusal regresyondakiyle aynıdır; yalnız çıkış sigmoid ile hesaplanır. Hoca bir sonraki derste, eğitmensiz öğrenmeye geçmeden önce özellik azaltma yöntemlerini anlatacağını; düzenlileştirme, bias ve variance konularına da derin öğrenme haftalarında yeniden döneceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.